Appscrip — Forward Deployed Engineer Assignment

Build a Real-Time Operations Platform

Suggested time: 8–12 hours

Submission: Working prototype, source code, and architecture notes

The Scenario

A client operates a fleet of devices across multiple locations. These devices continuously produce events, occasionally go offline, and may reconnect by sending a burst of previously buffered data.

The client wants a platform where operators can monitor device activity in real time, define rules for detecting important events, and trigger actions when those rules are satisfied.

For example:

- Alert an operator when a device stops reporting.

- Detect abnormal readings and trigger an incident.

- Notify the appropriate operator when a rule is triggered.

- Recover gracefully when devices or parts of the platform become unavailable.

The client has provided these requirements but has not specified the architecture, communication protocols, database, or deployment model.

Your job is to turn this vague requirement into a working solution.

1. Device and Event Ingestion

Build a mechanism for devices to send events to your platform.

Each event should contain enough information to identify its source, timestamp, type, and payload.

The system should account for:

- Events arriving out of order.

- Duplicate events.

- Devices disconnecting and reconnecting.

- Sudden bursts of incoming events.

- Invalid or malformed payloads.

You may define the event schema and device identity model.

2. Rules and Incident Management

Allow an operator to define rules for incoming events.

Examples include:

- Trigger an incident when a temperature reading exceeds a threshold.

- Trigger an incident if a device has not reported for a configurable duration.

- Trigger an incident when a device reports a specified event type.

When a rule matches, the system should create an incident that operators can inspect.

Incidents should have meaningful states, timestamps, and relevant event details. Repeated matching events should not create an uncontrolled number of duplicate incidents.

3. Live Operations Dashboard

Build a frontend where operators can:

- View registered devices and their current status.

- See incoming events without refreshing the page.

- Inspect incidents and their associated events.

- Acknowledge and resolve incidents.

- View recent activity and basic operational statistics.

The interface should remain useful when the connection is interrupted and subsequently restored.

4. Reliability and Failure Handling

The system must handle realistic failures.

Consider what happens when:

- The same event is delivered more than once.

- An event is accepted but processing fails.

- A worker or application process restarts.

- An external notification service is unavailable.

- A device reconnects after being offline for several minutes.

- Event ingestion temporarily exceeds processing capacity.

You do not need to solve every failure perfectly. You must explain your chosen guarantees, implement the most important ones, and document any remaining limitations.

5. Operational Requirements

Provide a way to understand what the system is doing.

Include useful logs, error visibility, basic health checks, and enough information to investigate why an incident was or was not created.

The system should be runnable locally with documented setup instructions.

6. Deliverables

Submit:

1. **Working application:** A functional frontend and backend, with any supporting services needed.

2. **Architecture diagram:** Show the major components and how data moves through the system.

3. **Architecture notes:** Explain your key decisions, alternatives considered, and trade-offs.

4. **Tests:** Demonstrate normal behavior and selected failure scenarios.

5. **Run instructions:** Make it straightforward for another engineer to start and evaluate your application.

6. **Deployment notes:** Explain how you would deploy, monitor, and scale the system for a real client.

7. What We Evaluate

We are interested in your ability to translate ambiguous requirements into a robust, useful product.

We will assess:

- Problem-solving and requirements interpretation.

- Architecture and engineering judgment.

- Reliability, concurrency, and data consistency.

- Backend and frontend implementation quality.

- Real-time communication and system integration.

- Observability and operational readiness.

- Practicality of deployment and scaling decisions.

- Clarity in communicating technical trade-offs.

There is no prescribed technology stack. Choose the tools and architecture you believe best solve the problem.

A smaller, reliable solution with well-reasoned decisions is preferable to a complicated system whose trade-offs are poorly understood.

Be prepared to explain not only how your system works, but also what happens when it fails.

Device Simulation and System Boundaries

You are expected to implement both the central platform and a lightweight device simulator. You do not need physical hardware or production-grade device firmware.

The simulator should demonstrate event generation, temporary disconnection, offline buffering, reconnection, and replay of buffered events.

You may implement the simulator in any language. Keep it lightweight; the primary focus is the central platform and the end-to-end system architecture.

Document the protocol and communication model you choose, including how your design handles duplicate events, delayed delivery, and device reconnection.